Assignment #1

Solving Nonlinear Equations

In this assignment students will learn how to compute the diode voltage using numerical root-finding algorithms.

Student Information

Run the cell below and enter your information.

In [67]:
```
from datetime import datetime
name = input("Enter your name: ")
student_id = input("Enter your student ID: ")
print("Student Name:", name)
print("Student ID:", student_id)
now = datetime.now()
print("Date:", now.strftime("%Y-%m-%d"))
print("Time:", now.strftime("%H:%M"))
```

```
Student Name: علاء الدين عبدالباسط نجاح
Student ID: 2220209198
Date: 2026-04-03
Time: 16:41
```

In [23]:
```
# ==============================
# Libraries used in this assignment
# ==============================
import numpy as np
# NumPy: used for numerical computations
# - provides mathematical functions
# - works efficiently with arrays
# - used for exp(), log(), linspace(), etc.
# - essential for implementing numerical methods
import matplotlib.pyplot as plt
# Matplotlib: used for plotting graphs
# - visualize functions
# - plot convergence curves
# - compare numerical methods
# - create figures such as error vs iteration
import pandas as pd
# Pandas: used for organizing results in tables
# - create structured data tables
# - display comparison of methods
# - export results to CSV or Excel if needed
```

Diode Circuit

The current through a diode follows the **Shockley diode equation**:

$$I = I_s(e^{V_D/V_T} - 1)$$

Using Kirchhoff's voltage law for the circuit:

$$\frac{V_{in} - V_D}{R} = I_s(e^{V_D/V_T} - 1)$$

This leads to the nonlinear equation:

$$f(V_D) = \frac{V_{in} - V_D}{R} - I_s(e^{V_D/V_T} - 1) = 0$$

Given Parameters

In [24]:
```
Vin = 5       # Input voltage (V)
R = 1000      # Resistance (Ohm)
Is = 1e-12    # Saturation current (A)
Vt = 0.025    # Thermal voltage (V)
```

Define the Nonlinear Function

Complete the function below

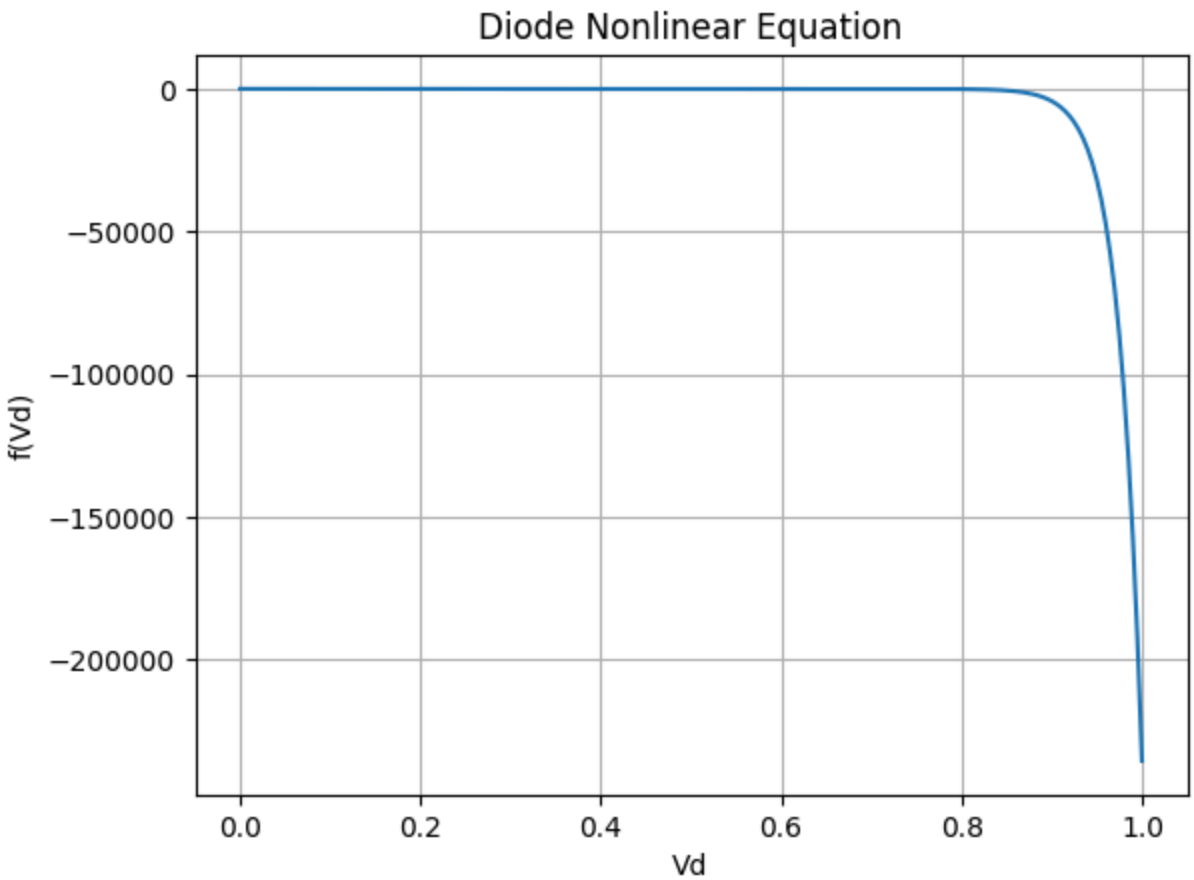
$$f(V_D) = \frac{V_{in} - V_D}{R} - I_s(e^{V_D/V_T} - 1) = 0$$

In [25]:
```
def f(Vd):
    return (Vin - Vd)/R - Is * (np.exp(Vd / Vt) - 1)
```

Task 1: Plot the Function

Plot the function in the interval **0 ≤ Vd ≤ 1** and estimate the root visually.

In [26]:
```
x = np.linspace(0,1,200)
y = f(x)
plt.plot(x,y)
plt.xlabel("Vd")
plt.ylabel("f(Vd)")
plt.title("Diode Nonlinear Equation")
plt.grid()
plt.show()
```



Implement a function for each method that returns the following

Output	Description
root	approximate solution of the nonlinear equation
iterations	number of iterations required to reach convergence
errors	list of relative errors at each iteration
order	estimated order of convergence

Task 2: Implement the false_position method

Parameter	Description
f	nonlinear function (f(x)) whose root is to be found
a	left endpoint of the interval (initial lower guess)
b	right endpoint of the interval (initial upper guess)

Parameter	Description
tol	tolerance value (stopping criterion based on error)
max_iter	maximum number of iterations allowed

Complete the missing parts of the algorithm.

In [27]:
```
def false_position(f, a, b, tol, max_iter):
    errors = []
    roots = []
    c_old = None  # Track the previous root estimate

    for i in range(max_iter):
        # 1. Calculate the new root estimate using the False Position formula
        fa = f(a)
        fb = f(b)
        c = (a * fb - b * fa) / (fb - fa)
        roots.append(c)

        # 2. Calculate Relative Approximation Error
        if c_old is not None and c != 0:
            rel_error = abs((c - c_old) / c)
            errors.append(rel_error)

            # 3. Stopping Criterion based on Relative Error
            if rel_error < tol:
                return c, i + 1, errors, get_order(errors)

        # 4. Update brackets
        fc = f(c)
        if fa * fc < 0:
            b = c
        else:
            a = c

        c_old = c  # Update for next iteration

    return c, max_iter, errors, get_order(errors)
```

Task 3: Implement the modified false_position method

In [56]:
```
def modified_false_position(f, a, b, tol, max_iter):
    errors = []
    roots = []

    fa = f(a)
    fb = f(b)
    c_old = None  # To track the previous estimate

    for i in range(max_iter):
```

```
        # 1. False position formula
        c = (a * fb - b * fa) / (fb - fa)
        roots.append(c)

        fc = f(c)

        # 2. Calculate Relative Approximation Error
        if c_old is not None and c != 0:
            rel_error = abs((c - c_old) / c)
            errors.append(rel_error)

            # 3. Stopping condition
            if rel_error < tol:
                return c, i + 1, errors, get_order(errors)

        # 4. Interval update
        if fa * fc < 0:
            b = c
            fb = fc
            fa = fa / 2
        else:
            a = c
            fa = fc
            fb = fb / 2

        c_old = c  # update for next iteration

    return c, max_iter, errors, get_order(errors)
```

Task 4: Implement Newton–Raphson Method

Hint: you need the derivative of the function.

In [57]:
```
def df(Vd):  # TODO: write derivative of the function.
    return -1/R - (Is/Vt) * np.exp(Vd / Vt)
```

Implements the Newton-Raphson method to find the root of a nonlinear equation and analyze convergence behavior. The function also computes: (relative approximation error, convergence history and order of convergence)

Parameter	Description
f1	the nonlinear function (f(x))
df	derivative of the function (f'(x))
x0	initial guess for the root
tol	tolerance (desired accuracy)
max_iter	maximum number of iterations allowed

In [58]:
```python
def newton(f, df, x0, tol, max_iter):
    errors = []
    x = x0

    for i in range(max_iter):
        # 1. Store the previous value for error calculation
        x_old = x

        # 2. Newton-Raphson step
        x_new = x - f(x)/df(x)

        # 3. Calculate Relative Approximation Error
        if x_new != 0:
            rel_error = abs((x_new - x) / x_new)
        else:
            rel_error = abs(x_new - x)

        errors.append(rel_error)

        # 4. Stopping Criterion
        if rel_error < tol:
            return x_new, i + 1, errors, get_order(errors)

        x = x_new

    # If max_iter reached without convergence
    return x, max_iter, errors, get_order(errors)
```

Task 5: Implement Secant Method

In [59]:
```python
def secant(f, x0, x1, tol, max_iter):
    errors = []

    for i in range(max_iter):
        f_x0 = f(x0)
        f_x1 = f(x1)

        # Avoid division by zero
        if f_x1 - f_x0 == 0:
            print("Division by zero in Secant formula")
            return x1, i, errors, get_order(errors)

        # Secant formula
        x2 = x1 - f_x1 * (x1 - x0) / (f_x1 - f_x0)

        # 2. Calculate Relative Approximation Error
        rel_error = abs((x2 - x1) / x2) if x2 != 0 else abs(x2 - x1)
        errors.append(rel_error)

        # Stopping criterion
        if rel_error < tol:
            return x2, i + 1, errors, get_order(errors)
```

```
        # Update points
        x0, x1 = x1, x2

    return x2, max_iter, errors, get_order(errors)
```

Don't modify the code of the following functions, Just run them and try to understand them

get_order(e) computes the order of convergence

store_results() stores the results in a table

convergence_comp() is designed to compare the convergence behavior of four root-finding numerical methods:

Compute Order of Converegence

In [60]:
```
def get_order(e): # compute order of convergence
    if len(e) < 3: return "N/A" # at least three errors
    p = np.log(e[-1]/e[-2]) / np.log(e[-2]/e[-3])
    return f"{p:.2f}"
```

Store the results in a table

In [61]:
```
def store_results():

# store results
    results = [
        {"Method": "False Position",
         "Root": root_fp,
         "Iterations": it_fp,
         "Final Error": err_fp[-1],
         "Order (p)": P_fp},

        {"Method": "Modified False Position",
         "Root": root_mfp,
         "Iterations": it_mfp,
         "Final Error": err_mfp[-1],
         "Order (p)": P_mfp},

        {"Method": "Newton",
         "Root": root_newton,
         "Iterations": it_newton,
         "Final Error": err_newton[-1],
         "Order (p)": P_n},
```

```
        {"Method": "Secant",
         "Root": root_sec,
         "Iterations": it_sec,
         "Final Error": err_sec[-1],
         "Order (p)": P_sec},
    ]

    # create table
    table = pd.DataFrame(results)
    return table
```

In [62]:
```
def convergence_comp():
    #It plots the relative approximation error at each iteration for all methods on
    # using a logarithmic y-axis to clearly visualize small errors.
    # errors can become extremely small after a few iterations. On a linear scale,
    # appear as zero, making it hard to visualize the convergence.
    plt.figure(figsize=(10, 6))
    plt.plot(range(1, len(err_newton)+1), err_newton, label=f"Newton (p={get_order(
    plt.plot(range(1, len(err_sec)+1), err_sec, label=f"Secant (p={get_order(err_se
    plt.plot(range(1, len(err_mfp)+1), err_mfp, label=f"Mod-False Pos (p={get_order
    plt.plot(range(1, len(err_fp)+1), err_fp, label=f"False Pos (p={get_order(err_f

    plt.yscale('log') # to change the y-axis scale from linear to logarithmic.

    plt.grid(True, which="both", ls="-", alpha=0.3)
    plt.title("Convergence Speed Comparison")
    plt.xlabel("Iteration")
    plt.ylabel("Relative Approximation Error")
    plt.legend()
    plt.show()
    return
```

Task6: Run the four Methods with the following parameters

Tolerance value = 1e-6, max_iter =100

Initial Interval of false position method is [0,0.4]

Initial guess for the root (using Newton Method)= 0.4

for Secant method let x0=0.4 and x1=0.8

In [63]:
```
root_fp, it_fp, err_fp, P_fp = false_position(f, 0, 1, 1e-6, 100)

root_mfp, it_mfp, err_mfp, P_mfp = modified_false_position(f, 0, 1, 1e-6, 100)

root_newton, it_newton, err_newton, P_n = newton(f, df, 0.7, 1e-6, 100)

root_sec, it_sec, err_sec, P_sec = secant(f, 0, 1, 1e-6, 100)
```

```
table = store_results()
table
```

```
C:\Users\Ahmed\AppData\Local\Temp\ipykernel_15548\3523202275.py:3: RuntimeWarning: d
ivide by zero encountered in log
  p = np.log(e[-1]/e[-2]) / np.log(e[-2]/e[-3])
```

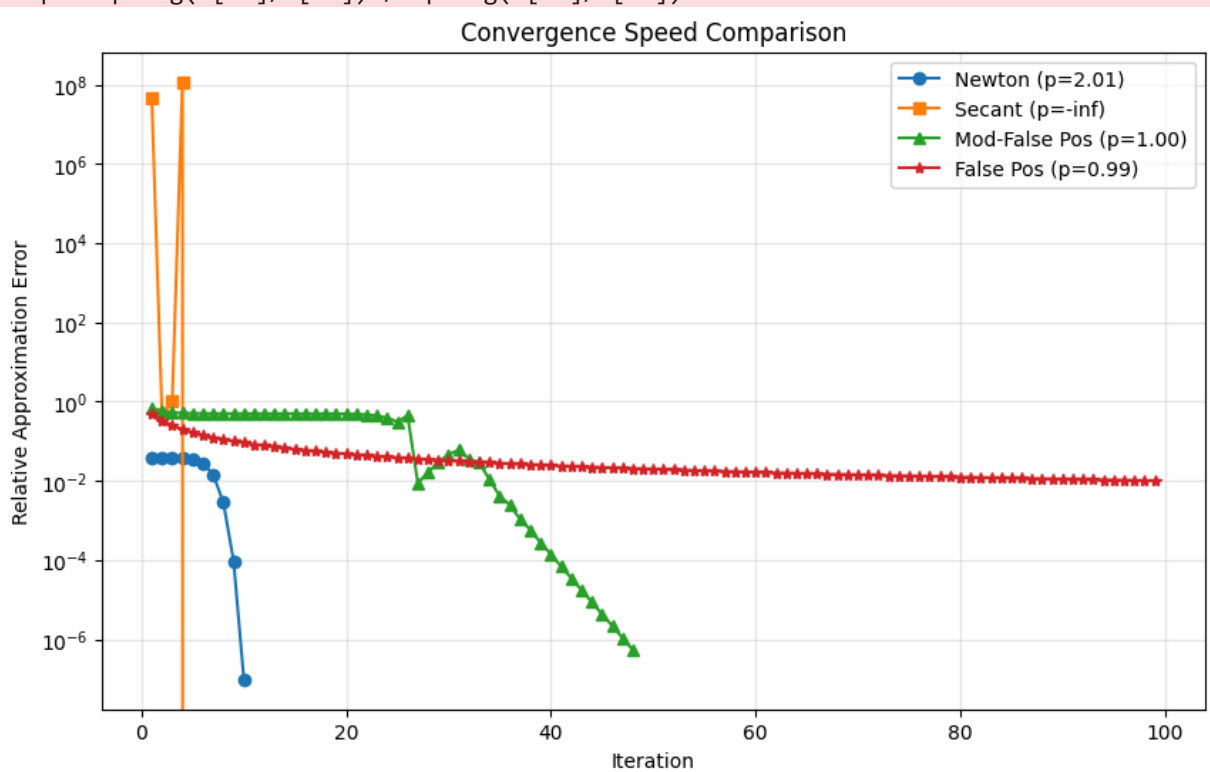
Out[63]:

	Method	Root	Iterations	Final Error	Order (p)
0	False Position	2.124174e-06	100	9.999987e-03	0.99
1	Modified False Position	5.553741e-01	49	5.428083e-07	1.00
2	Newton	5.553740e-01	10	9.742307e-08	2.01
3	Secant	4.248354e-08	5	0.000000e+00	-inf

In [64]:

```
convergence_comp()
```

```
C:\Users\Ahmed\AppData\Local\Temp\ipykernel_15548\3523202275.py:3: RuntimeWarning: d
ivide by zero encountered in log
  p = np.log(e[-1]/e[-2]) / np.log(e[-2]/e[-3])
```



Task 7: Compare Methods

Discuss:

- Which method converges faster? Which method is more stable?
- Add your answer

Run the method with different parameters and compare the results with the previous ones

Tolerance value = 1e-6, max_iter =50

Initial Interval of false position method is [0,1]

Initial guess for the root (using Newton Method)= 0.7

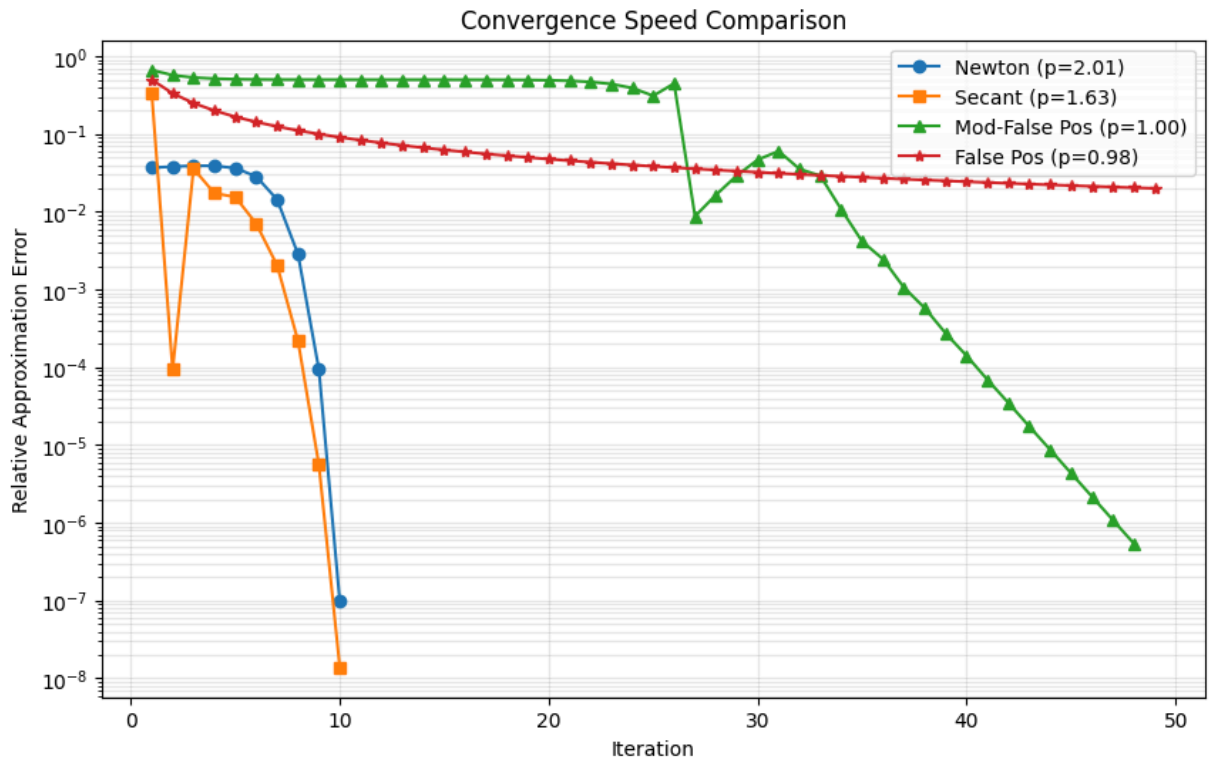
for Secant method let x0=0.6 and x1=0.8

In [65]:
```
tol = 1e-6
max_iter = 50

root_fp, it_fp, err_fp, P_fp = false_position(f, 0, 1, tol, max_iter)

root_mfp, it_mfp, err_mfp, P_mfp = modified_false_position(f, 0, 1, tol, max_iter)

root_newton, it_newton, err_newton, P_n = newton(f, df, 0.7, tol, max_iter)

root_sec, it_sec, err_sec, P_sec = secant(f, 0.6, 0.8, tol, max_iter)

# Store results in your table
table = store_results()
table
```

Out[65]:

	Method	Root	Iterations	Final Error	Order (p)
0	False Position	0.000001	50	1.999999e-02	0.98
1	Modified False Position	0.555374	49	5.428083e-07	1.00
2	Newton	0.555374	10	9.742307e-08	2.01
3	Secant	0.555374	10	1.384110e-08	1.63

In [66]:
```
convergence_comp()
```



Task 8 Discuss the results

From the results, we can see clear differences in convergence speed and stability among the methods:

Newton's Method converged the fastest, reaching the solution in about 10 iterations with a very small error. Its order of convergence was close to 2, which confirms its quadratic convergence property.

Secant Method also performed well, converging in 10 iterations with an order of convergence around 1.6. It is slightly slower than Newton but does not require the derivative, making it more practical in some cases.

Modified False Position Method converged to the correct root but required many more iterations (49). Its order of convergence was about 1, showing linear behavior.

Standard False Position Method was the slowest and least stable, stuck near a very small root with poor accuracy even after 50 iterations.

Conclusion: Newton's method is the most efficient and stable when a good initial guess and derivative are available. The Secant method is a good alternative when derivatives are difficult to compute. False Position methods are more robust but converge slowly.

Task 9

Students are encouraged to experiment with the root-finding methods by using parameter values of their choice, such as:

Initial guesses (x0 for Newton/Secant, [a,b] for False Position) Tolerance (tol) Maximum iterations (max_iter)

1- Explore convergence behavior

```
Changing the initial guess strongly affects Newton and Secant
methods. A poor guess can slow convergence or even cause
divergence.

False Position methods are less sensitive to initial guesses since
they always keep the root bracketed,  but they converge more
slowly.
```

2- Compare method efficiency

```
Newton’s method usually requires fewer iterations due to quadratic
convergence.

Secant is efficient but slightly slower.

False Position methods need many iterations, especially the
standard version, which makes them less efficient..
```

3- Learn robustness

```
Newton may fail if the derivative is very small or the initial
guess is far from the root.

Secant can also fail if the two starting points are not chosen
carefully.

False Position methods are more robust because they maintain a
bracket, but this comes at the cost of speed.
```